

python.builtin#

<https://pytorch.org/docs/stable/generated/exportdb/python.builtin.html>

Description:

Tags: torch.dynamic-shape, python.builtin Support Level:
NOT_SUPPORTED_YET Original source code:

Code Examples

```

# mypy: allow-untyped-defs
import torch

from torch._export.db.case import SupportLevel
from torch.export import Dim

class DynamicShapeRound(torch.nn.Module):
    """
    Calling round on dynamic shapes is not supported.
    """

    def forward(self, x):
        return x[: round(x.shape[0] / 2)]

x = torch.randn(3, 2)
dim0_x = Dim("dim0_x")
example_args = (x,)
tags = {"torch.dynamic-shape", "python.builtin"}
support_level = SupportLevel.NOT_SUPPORTED_YET
dynamic_shapes = {"x": {0: dim0_x}}
model = DynamicShapeRound()

torch.export.export(model, example_args,
dynamic_shapes=dynamic_shapes)

```

Unsupported: Constraints violated (dim0_x)! For more information, run with TORCH_LOGS="+dynamic".

```
# mypy: allow-untyped-defs
import torch

class TensorSetattr(torch.nn.Module):
    """
    setattr() call onto tensors is not supported.
    """
    def forward(self, x, attr):
        setattr(x, attr, torch.randn(3, 2))
        return x + 4

example_args = (torch.randn(3, 2), "attr")
tags = {"python.builtin"}
model = TensorSetattr()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 2]", attr):
        randn: "f32[3, 2]" =
torch.ops.aten.randn.default([3, 2], device =
device(type='cpu'), pin_memory = False); randn = None

        add: "f32[3, 2]" = torch.ops.aten.add.Tensor(x,
4); x = None
        return (add,)
```

Graph signature:

```
# inputs
x: USER_INPUT
attr: USER_INPUT

# outputs
add: USER_OUTPUT
```

Range constraints: {}

```
# mypy: allow-untyped-defs
import torch

class A:
    @classmethod
    def func(cls, x):
        return 1 + x

class TypeReflectionMethod(torch.nn.Module):
    """
    type() calls on custom objects followed by attribute
    accesses are not allowed
    due to its overly dynamic nature.
    """

    def forward(self, x):
        a = A()
        return type(a).func(x)

example_args = (torch.randn(3, 4),)
tags = {"python.builtin"}
model = TypeReflectionMethod()

torch.export.export(model, example_args)
```

```
ExportedProgram:
  class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 4]"):
      add: "f32[3, 4]" = torch.ops.aten.add.Tensor(x,
1);  x = None
      return (add,)

Graph signature:
  # inputs
  x: USER_INPUT

  # outputs
  add: USER_OUTPUT

Range constraints: {}
```

Notes & Warnings

NOTE Note Tags: torch.dynamic-shape, python.builtin Support Level: NOT_SUPPORTED_YET

NOTE Note Tags: python.builtin Support Level: SUPPORTED

NOTE Note Tags: python.builtin Support Level: SUPPORTED

Detailed Documentation

dynamic_shape_round#

Original source code:

tensor_setattr#

Original source code:

type_reflection_method#

Original source code:

Tags: torch.dynamic-shape, python.builtin

Support Level: NOT_SUPPORTED_YET

Original source code:

Support Level: SUPPORTED

Original source code:

Support Level: SUPPORTED

Original source code:
